



Find. Build. Collaborate.

CollabX is a console-based project collaboration platform written in Java. It gives students a single place to publish project ideas, discover projects that suit their skills, apply to join a team, and then run the project through simple task assignment and status tracking. The tagline that runs through the interface — find, build, collaborate — describes the three phases the software supports: discovery, team formation, and execution.

The application was built as part of a Java and object-oriented programming course, so it has two audiences at once. For a student user, it is a working tool that solves a real coordination problem. For the course, it is a demonstration of classes and objects, encapsulation of related state and behaviour, the Java Collections Framework, menu-driven control flow, defensive input handling, and a small piece of algorithmic logic in the form of a skill-matching score.

The current version runs entirely in memory. There is no database, no network layer, and no graphical interface. Every user, project, application and task lives in an ArrayList for the duration of the session and is discarded when the program exits. This is a deliberate scope choice rather than an oversight: the goal of the prototype is to prove that the domain model and the workflow are correct and coherent, so that persistence and a user interface can later be layered on without redesigning the core. The report describes the system as it stands, explains why it was built this way, and sets out what a second version would change.

Problem Statement

Students with project ideas often cannot pursue them alone, and those who learn new skills rarely find opportunities to apply them. A student with Python knowledge might want to build a machine learning traffic prediction prototype but lack a teammate for the UI. Meanwhile, a student with React experience has no way to discover projects needing front-end help. Informal channels exist but have three key limitations:

Limited Reach

Opportunities are confined to a student's immediate social circle.

No Shared Understanding

No clear record of what a project requires or what a student can contribute.

No Team Tracking

No way to track who does what after a team is formed.

CollabX addresses each issue: projects list required skills and team size, students maintain skill records for matching, applications carry status fields (PENDING / ACCEPTED / REJECTED), and task management tracks progress. Scope is limited to project discovery, application management, and task assignment — messaging, grading, and code hosting are left to other tools.

Functional Requirements

Seven requirement groups define CollabX's behavior across the full user journey.

1

User Management

Register with name, email, password, course, and comma-separated skills (normalized to lowercase). Unique integer ID assigned on registration. Login via case-insensitive email; generic error on failure. View profile and logout.

2

Project Management

Create projects with title, description, category, team size, and required skills. Unique ID assigned; status set to OPEN. Creator added as first team member. All users (including visitors) can browse projects.

3

Search & Recommendation

Keyword search across title, category, description, and skills (case-insensitive, partial matches). Skill match = $\frac{\text{matching skills}}{\text{required skills}} \times 100$. Recommendations show projects with at least one match and an open slot.

4

Applications

Apply to a project by ID with a message. Blocked if project invalid, full, or already applied/joined. Status: PENDING (default), ACCEPTED, or REJECTED. Users can view all their applications.

5

Team Management

Creators view their projects and all applications. Accepting an application adds the applicant to the team and updates status. Team size limit enforced. Creators can view team members and their skills.

6

Task Management

Creators create tasks with title, description, assignee, and deadline. Assignment limited to project team members. Task status: PENDING, IN PROGRESS, or COMPLETED. Users can view and update their tasks.

7

Input Handling

Invalid numeric inputs re-prompt without terminating. Invalid ID lookups show an error and return to menu. Unrecognized menu selections display an error and repeat the menu.

Non-Functional Requirements



Usability

Menu-driven interface with banners, option lists, and prompts. No irreversible actions — data changes require re-entry. Input validation catches errors before they propagate.



Reliability

Invalid inputs never crash the app — prompts repeat or errors display. Only `exit` terminates the program. All ID lookups validate before dereferencing.



Performance

Linear scans over `ArrayList` instances — no database or network I/O. Appropriate for tens of students; the focus is on the domain model, not premature optimization.



Maintainability

Logical method separation by purpose, Java naming conventions, organized by concern. Entire prototype fits in one file with no framework beyond `Scanner` and `ArrayList`.



Portability

Uses only `java.util` classes. Launchable from any text editor on any machine with a JDK — no build tools or special runtime environments required.



Security

Plain-text passwords and coarse session management are acceptable for a low-value prototype but will be replaced with encrypted storage and session tokens in production.

System Architecture

CollabX uses a conceptual three-layer architecture within a single file. The presentation layer never touches collections directly — every read and write passes through a named operation, enabling clean migration to proper separation of concerns in future iterations.

Presentation Layer

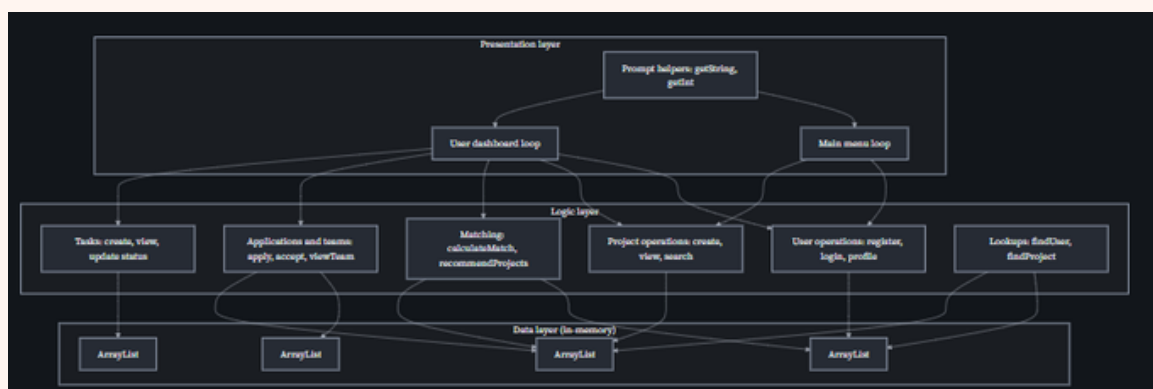
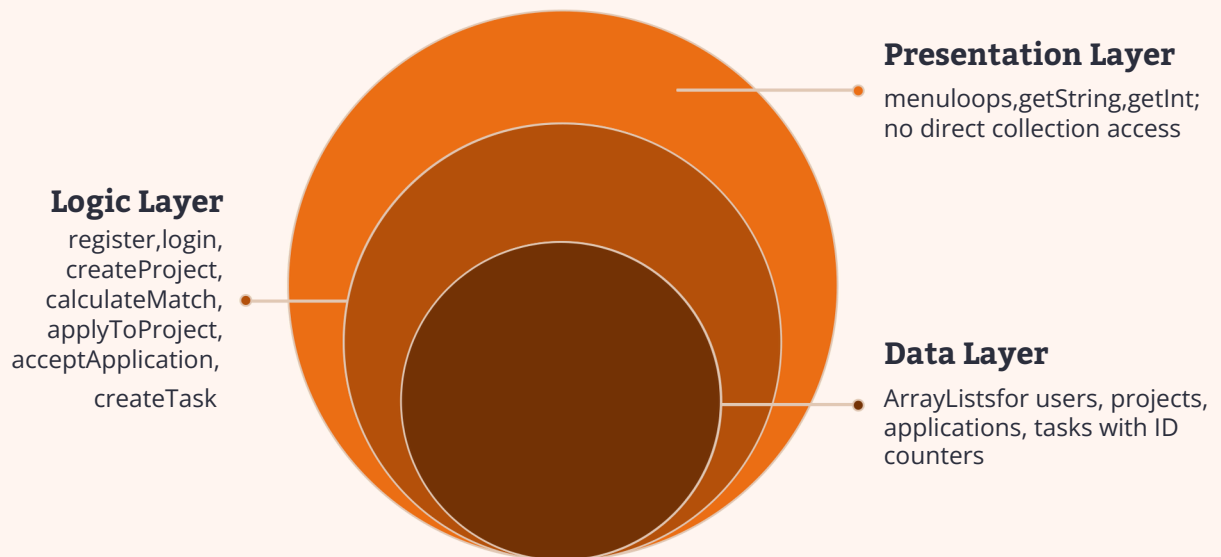
Owns all console I/O. Implements the public menu loop, authenticated dashboard, sub-menus, and helper methods (getString, getInt). Every other method invokes these helpers rather than `Scanner` directly.

Logic Layer

Implements all use cases. Methods belong to domain classes or are stateless single-use-case actions (e.g., `calculateMatch`). Every conditional branch includes a comment explaining its purpose.

Data Layer

Four static `ArrayList` instances (users, projects, applications, tasks) plus counter variables for unique ID allocation. Domain objects hold foreign keys as integers rather than direct object references — easier to migrate to a relational database.



Design Diagrams

Five diagrams document CollabX's structure from actor interactions through to the proposed database schema.

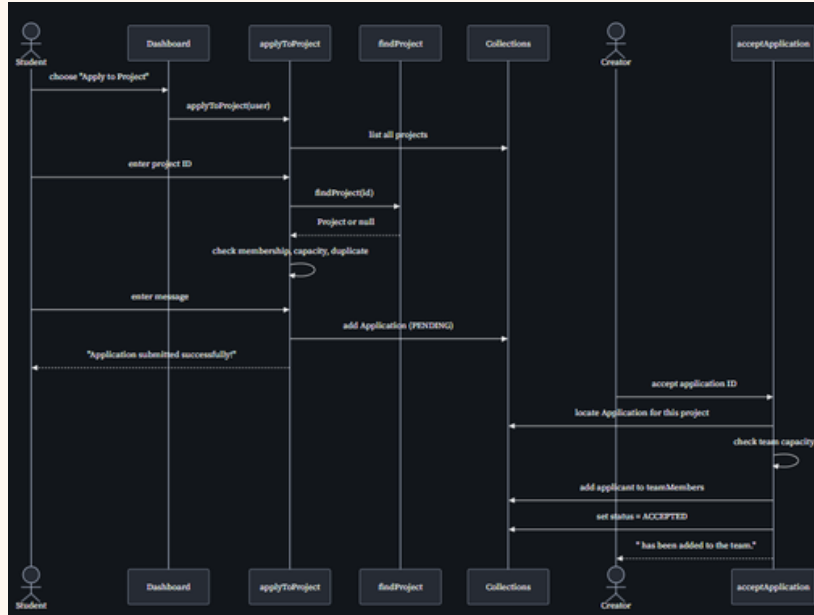
Case Diagram:



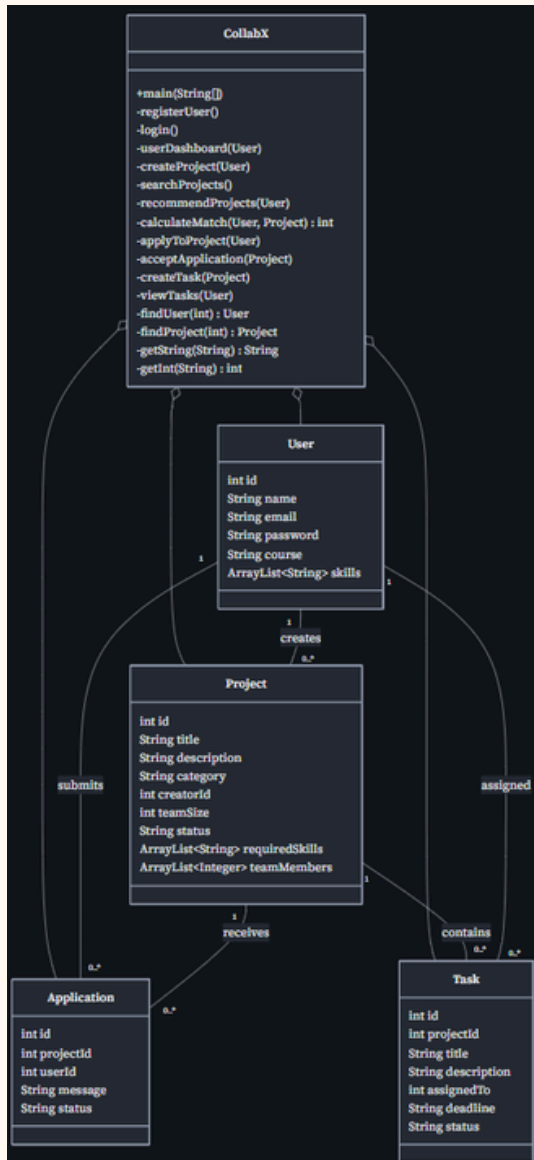
Workflow Diagram:



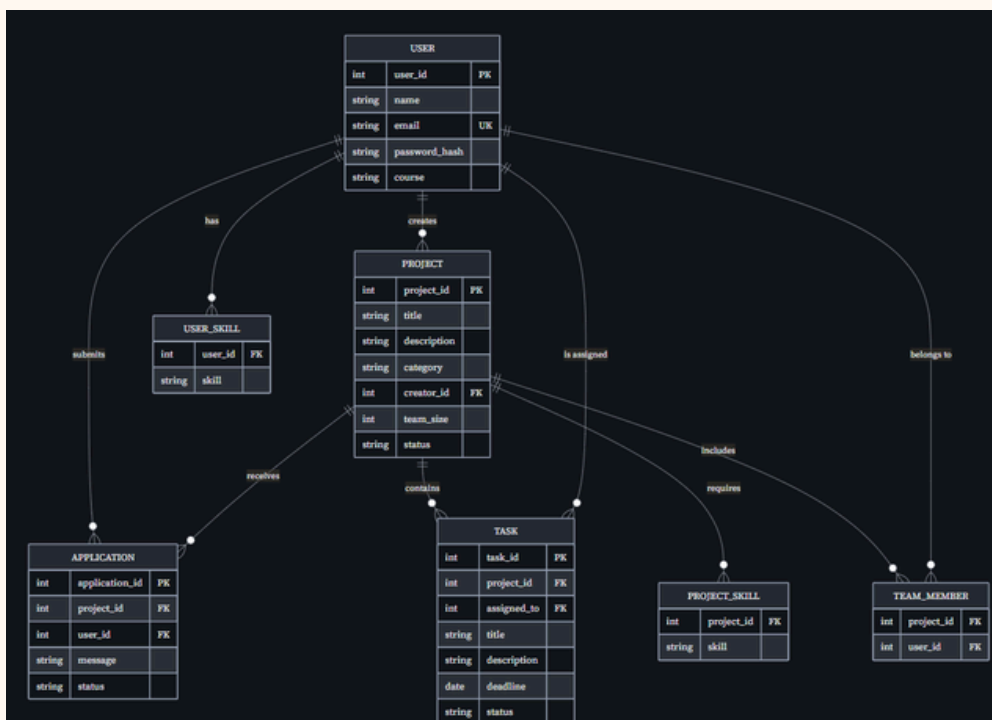
Sequence Diagram:



Class Diagram:



ER Diagram:



Design Decisions & Implementation

References by ID, Not Object

Projects store user IDs, not `User` object references, avoiding duplication when the same user appears in multiple projects. Skills are normalized to lowercase at input, preventing "Java" vs "java" mismatches.

Percentage Match Algorithm

$\text{Match} = \text{matching skills} \div \text{required skills} \times 100$. Integer division avoids floating-point issues: 2 of 3 skills = 66%. Projects with 0% match or no open slots are filtered from recommendations.

Guarded Input & Layered Validation

`getInt` reprompts until valid — typos don't crash the app. `applyToProject` checks: project exists not already a member → team not full → no pending duplicate → then accepts the application message.

Seeded Sample Data

`addSampleData()` pre-populates three users with varying skills and two projects before the main menu, enabling immediate demonstration of matching without manual registration.

Control Flow

Two nested loops: outer loop shows the public menu; inner loop runs the authenticated dashboard. The logged-in `User` is passed as an argument — no separate session object needed.

Results & Screenshots

The application was run through its seeded dataset. Key console outputs confirm core functionality works end-to-end.

Skill-Based Recommendations

Logged in as **Rahul** (skills: java, sql, python), the recommendation screen shows:
Project: Smart Traffic Prediction
Skill Match: 66%
Category: Machine Learning
Required Skills: python, machine learning, sql
Two of three required skills match — 66% correctly calculated, not rounded to 0%. Campus Marketplace (33%) and a zero-match project also appear in the list.

Application & Team Formation

Application submission returns "**Application submitted successfully!**" After the creator accepts, status changes from PENDING to ACCEPTED and the new member appears in the team skills view.

Guard Behavior Verified

- Applying twice → "**You have already applied.**"
- Accepting beyond team size → "**Team is already full.**"
- Letters at numeric prompt → re-prompts, no crash

Test Results Summary

ID	Test Case	Result
T1-T4	Register, login, create project	✔ Pass
T5-T7	Search and recommendations	✔ Pass
T8-T10	Apply guards (invalid ID, duplicate, already member)	✔ Pass
T11-T13	Team size limit, task assignment, status update	✔ Pass
T14	Letters at numeric prompt	✔ Pass

```
=====
          COLLABX PLATFORM
    Find. Build. Collaborate.
=====
```

1. Register
2. Login
3. View Projects
4. Search Projects
5. Exit

Enter choice:

```
===== AVAILABLE PROJECTS =====
```

```
-----
Project ID: 1
Title: Smart Traffic Prediction
Category: Artificial Intelligence
Description: An ML system that predicts traffic congestion.
Team: 1/4
Required Skills: python, machine learning, sql
Status: OPEN
```

```
-----
Project ID: 2
Title: Campus Marketplace
Category: Web Development
Description: Platform for students to buy and sell used products.
Team: 1/4
Required Skills: react, javascript, sql
Status: OPEN
```

Application submitted successfully!

...

Neha has been added to the team.

```
===== LOGIN =====
```

Email: rahul@gmail.com

Password: ****

Welcome, Rahul!

Challenges Faced & Key Learnings

Challenges Faced

- **Integer division bug:** `(matchingSkills / requiredSkills.size()) * 100` returned 0 for all partial matches. Fixed by reordering: `(matchingSkills * 100) / requiredSkills.size()`.
- **Skill normalization:** "Java, SQL" didn't match "java, sql". Fixed by trimming and lowercasing at input rather than during comparison.
- **Scanner buffer issue:** `nextInt()` left a newline consumed by subsequent `nextLine()`. Replaced with a general-purpose `getLine()` that reads the full line and parses it.
- **Email duplication:** Registration does not check email uniqueness — two users with the same email can log in. Acknowledged open issue (T15).
- **Deferred shortcuts:** Package-private fields instead of private with getters/setters, and deadlines stored as strings rather than `LocalDate` — marked low priority.

Key Learnings

- **Data model drives everything:** Choosing ID references over object references kept the model simple and deferred complexity to the controller.
- **Validate at the boundary:** Normalizing skills once at input eliminated repeated comparison logic throughout the codebase.
- **Separation of concerns matters:** Even within one file, keeping presentation, logic, and data separate means each layer can change independently.
- **Test against requirements:** Test cases extracted from the requirements document surfaced the duplicate-email bug that code-level tests would have missed.

Learnings & Key Takeaways

A retrospective analysis of the project's development revealed several critical lessons, impacting everything from data structure to testing methodologies. These insights are invaluable for future project undertakings and reinforce best practices in software development.

The key lessons learned during the CollabX project included:

Data Modelling Foundation

The strategic decision to use ID-based references instead of direct object references in the data model was pivotal. This choice simplified complex aspects like team membership and application management, and allowed for a clear derivation of a relational schema. While potentially slower to initially implement, it significantly eased future migration and maintained a clean, scalable model.

Separation of Concerns

Adhering to the separation of presentation, business logic, and data access, even within a single file, yielded significant benefits. This conventional distinction, alongside centralizing all read operations through dedicated helper functions, allowed package splits and input hardening efforts to be localized, single changes rather than requiring extensive, error-prone rewrites across the codebase.

Foundary Validation

Implementing data validation at the input boundary, such as normalizing skill strings immediately upon entry, proved highly effective. This approach transformed recurring comparison bugs into non-issues by ensuring data consistency at its source. It underscored the principle that enforcing invariants is most efficient when data first enters the system, preventing errors from propagating.

Requirement Driven Testing

The project vividly demonstrated the value of testing against explicit requirements rather than against the implemented code. By deriving test cases directly from the functional requirement list, a critical missing duplicate-email check was identified. This bug might have been overlooked if testing had solely focused on verifying existing code behavior, highlighting the importance of an independent, requirement-centric testing strategy.

Future Enhancements & References

Immediate Fixes

Prevent duplicate email registration; add email format validation. Make model fields private with getters/setters. Store deadlines as `LocalDate` and add overdue-task checks.

Security & Matching

Hash passwords with BCrypt or PBKDF2; add complexity requirements. Weight skills by proficiency; distinguish required vs. optional skills in match calculation. Rank recommendations by match percentage, category, and availability.

1

2

3

4

Code Organization

Separate model, service, and utility classes into their own packages. Implement the ER schema on MySQL via JDBC and DAO interfaces. Explore JSON/CSV file persistence as a lighter alternative.

Interface & Integrations

Add in-platform messaging, task comments, and file attachments. Migrate to JavaFX or Spring Boot with REST API. Integrate GitHub repositories for completion tracking and team formation analytics.

References

- Oracle. *Java Platform SE 17 API Specification* — ArrayList, Scanner, LocalDate. Oracle. *The Java Tutorials: Collections Trail*.
- Bloch, Joshua. *Effective Java, 3rd ed.* Addison-Wesley, 2018. Schildt, Herbert. *Java: The Complete Reference, 12th ed.* McGraw-Hill, 2021.
- Martin, Robert C. *Clean Code*. Prentice Hall, 2008. Fowler, Martin. *UML Distilled, 3rd ed.* Addison-Wesley, 2003.
- Elmasri & Navathe. *Fundamentals of Database Systems, 7th ed.* Pearson, 2016. Oracle. *JDBC Basics*. OWASP. *Password Storage Cheat Sheet*.
- CollabX project repository — CollabX.java, README.md, project statement.